



Mission: The Talking Smart Dice

Goal: Turn your micro:bit into a smart dice that rolls when shaken, plays music, and speaks the result.

💡 Scaffolding: How to Build Your Code

Follow this structure to organize your logic. Think of it like building a tower—the foundations (imports) must come first!

1. The Setup (Imports): Tell the micro:bit which "tools" you need (random, music, and speech).
2. The Loop: Use while True: to keep the micro:bit "listening" for your movement.
3. The Trigger: Use an if statement combined with or to check for a shake or a button press.
4. The Action:
 1. Pick a random number between 1 and 6.
 2. Play a melody.
 3. Show the number on the LED screen.
5. The Personality: Use if/elif statements to check what number was rolled and have the speech module say a custom phrase for that number.

Starter code:

```
# Imports go at the top
from microbit import *
import random

# step one
while True:
    roll = random.randint(1,6)
    display.show(roll)
    sleep(2000)
```

Phase 1: (The Basics)

Before we add new features, let's review how the basic dice works.

1. To get a random number, we must import _____ at the very top of our code.
 2. The numbers are shown on the **5x5 grid** of red lights called the _____.
 3. To make the code run forever and wait for us, we use a _____ True: loop.
 4. The command `random.randint(1, 6)` picks a number between _ and _.
-

Phase 2: (Adding Motion)

The dice currently rolls constantly. We want it to roll only when we move the micro:bit.

1. The sensor that detects movement and tilt is called the _____.
 2. To check if the device was moved quickly, we check for a gesture called a _____.
 3. To make the dice roll *only* when moved, we put our code inside an _____ statement.
 4. **Logic Check:** If the micro:bit is "still," the dice should (do nothing / keep rolling).
-

Phase 3: (Adding Sound)

Let's give the dice a "voice" using music.

1. To use music, you must add `import` _____ at the top of your script.
 2. Sound comes out of the built-in _____ on the back of the micro:bit.
 3. To play a built-in melody like 'JUMP_UP', you use the command `music. _____`.
 4. You should put the music command (before / after) the number appears on the screen so we hear it first.
-

Phase 4: (Talking Dice)

We want the micro:bit to speak the number it rolled. Different numbers should have different "personalities."

1. The module used to make the micro:bit speak English words is called _____.
 2. If the random number variable is `x`, we use the command `speech. _____ ("Hello")` to hear it.
 3. To give different numbers different sounds, we use **Conditional Logic**:
 - `if x == 1: -> Say "You got a tiny one!"`
 - _____ `x == 6: -> Say "BULLSEYE! SIX!"`
 - `else: -> Say the number normally.`
 4. In Python, we use a double equals sign _____ to check if a variable matches a number.
-

Phase 5: (The Backup Button)

Sometimes shaking is too much work. Let's add a button backup.

1. The two physical inputs on the front are `button_a` and _____.
2. We want the dice to roll if we **Shake it** _____ if we **Press Button A**.
3. The keyword used to join two choices together so that *either* one works is _____.
4. **Final Logic Goal:** If (Shake) _____ (Button A is pressed) _____ (Button B is pressed): **Roll Dice!**